

[Skip to content](#)

Chat history

You said:

Explain the difference between SGD, momentum, and Adam in a way that emphasizes intuition rather than formulas.

EECS182 Helper said:

Here's a conceptual, geometry-first way to distinguish **SGD**, **momentum**, and **Adam**, the way we'd talk about them in EECS182 discussion.

□□ **SGD: “Walk straight downhill.”**

Intuition:

Imagine you're standing on a hilly landscape (the loss surface) and you want to reach the bottom. SGD looks at the slope right under your feet and takes a small step downhill.

-

It's **reactive** — every step depends only on the current slope.

-

If the terrain is noisy (mini-batches!), SGD jitters around a lot.

-

If one direction is steep and another is flat, SGD may zig-zag or make painfully slow progress in the flat direction.

Local linear view: SGD trusts the local tangent plane completely, with no memory of what happened before.

☐☐☐ Momentum: “Carry your velocity.”

Intuition:

Still walking downhill, but now imagine you have **mass**.

Every time you step, you don't just respond to the current slope — you also keep some of your previous motion.

-

Momentum **smooths out noise**: small inconsistent slopes cancel, persistent slopes accumulate.

-

It reduces zig-zagging: if the slope oscillates left-right but consistently points forward, momentum pushes you forward.

-

Think of it as **low-pass filtering** the gradient.

Local linear view: momentum is trying to approximate the fact that the optimal direction often doesn't change abruptly — so it averages local linear information over time.

I sometimes call momentum an “**inertia amplifier for consistent gradients.**”

□□ **Adam: “Adjust your step size per direction.”**

Intuition:

Now imagine the ground under you changes in texture. Some directions feel squishy (large variance gradients), others firm (predictable gradients). You'd want to take **smaller steps in squishy directions** and **bigger steps in firm ones**.

Adam does two things:

- 1.

Keeps a momentum-like running average of gradients □ direction smoothing.

- 2.

Tracks how variable each coordinate's gradients are □ step-size adjustment.

This per-coordinate adaptation gives Adam a kind of **built-in learning rate schedule** that treats each

dimension independently.

- If a coordinate’s gradients bounce around, Adam shrinks steps in that direction.
- If gradients are stable, Adam takes bolder steps.
- This is why Adam often converges faster in practice, especially in problems with very different curvature scales across parameters (common in deep nets).

Local linear view: Adam is trying to adjust the “units” of each direction so that each coordinate behaves more like a well-conditioned direction in a locally linear approximation.

□ A simple metaphor to tie them together

Optimizer	Metaphor	What it “believes” about the landscape
SGD	Walker taking steps based only on the slope underfoot	Local slope is reliable; past doesn’t matter
Momentum	A heavy ball rolling downhill	Gradient direction changes slowly; noise should be smoothed
Adam	A self-adjusting shoe that shortens/lengthens steps depending on terrain variability	Landscape is anisotropic; each direction deserves its own step size

□ A quick diagnostic question for you

If you had a loss landscape shaped like a very narrow valley — steep walls but a long flat direction — which optimizer do you think would escape the zig-zagging and move efficiently along the valley floor? **Why?**

(Your answer will help me tailor the next explanation.)

You said:

Gradient Descent Doesn't Go Nuts with Ill-Conditioning Consider a linear regression problem with n

training points and d features. When $n = d$, the feature matrix $F \in \mathbb{R}^{n \times n}$ has some maximum singular value $\sigma_{\max}$ and an extremely tiny minimum singular value. We have noisy observations $y = F w^* + \epsilon$. If we compute $\hat{w} = F^{-1}y$, then due to the tiny singular value of F and the presence of noise we observe that $\|\hat{w} - w^*\|^2 = 10^{10}$. Suppose instead of inverting the matrix we decide to use gradient descent instead. We run k iterations of gradient descent to minimize the loss $\ell(w) = \frac{1}{2} \|y - F w\|^2$ starting from $w_0 = 0$. We use a learning rate η which is small enough that gradient descent cannot possibly diverge for the given problem. (This is important. You will need to use this.) The gradient-descent update for $t > 0$ is: $w_t = w_{t-1} - \eta \nabla \ell(w_{t-1}) = w_{t-1} - \eta F^T (F w_{t-1} - y)$. We are interested in the error $\|w_k - w^*\|^2$. We want to show that in the worst case, this error can grow at most linearly with iterations k and in particular $\|w_k - w^*\|^2 \leq k \sigma_{\max}^2 \|\epsilon\|^2 + \|w^*\|^2$. i.e. The error cannot go “nuts,” at least not very fast. For the purposes of the homework, you only have to prove the key idea, since the rest follows by applying induction and the triangle inequality. help me solve this

EECS182 Helper said:

Thought for 1m 30s

Nice, this is a really important intuition question: why does GD not completely explode even when $F - 1F - 1$ is a disaster?

Let me walk you through the **key idea** step-by-step and stop just short of a full written solution so you still get credit for the derivation.

1. Re-express the update in terms of the error

Define the error at step t :

$$e_t = w_t - w^* \quad e_t = w_t - w^*$$

You're minimizing

$$J(w) = \frac{1}{2} \|y - Fw\|^2, y = Fw^* \quad J(w) = \frac{1}{2} \|y - Fw\|^2, y = Fw^*$$

Gradient descent update:

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \mathbf{F}(\mathbf{F} \mathbf{w}_{t-1} - \mathbf{y})$$

Plug in $\mathbf{y} = \mathbf{F} \mathbf{w}^* + \epsilon$:

$$\begin{aligned} \mathbf{w}_t &= \mathbf{w}_{t-1} - \eta \mathbf{F}(\mathbf{F} \mathbf{w}_{t-1} - \mathbf{F} \mathbf{w}^* - \epsilon) \\ &= \mathbf{w}_{t-1} - \eta \mathbf{F}(\mathbf{F}(\mathbf{w}_{t-1} - \mathbf{w}^*) - \epsilon) \end{aligned}$$

Now subtract $\mathbf{w}^*$ from both sides to get an update for $\mathbf{e}_t$:

$$\mathbf{e}_t = \mathbf{w}_t - \mathbf{w}^* = (\mathbf{I} - \eta \mathbf{F} \mathbf{F}) \mathbf{e}_{t-1} + \eta \mathbf{F} \epsilon_t$$

□ This is the **key structural equation**:

error at step t = “shrunk old error” + “noise-driven term”

2. Use the “no divergence” condition

We’re told:

“We use a learning rate η which is small enough that gradient descent cannot possibly diverge.”

For a quadratic loss with Hessian $\mathbf{F} \mathbf{F} \mathbf{F}$, this means:

$$\|2\|_{\max}(F^T F) = 2\|2\|_{\max}(F^T F)^2 = \|2\|^2$$

The eigenvalues of $F^T F F^T F$ are in $[0, \|2\|^2]$, so the eigenvalues of

$$I - \eta F^T F I - \eta F^T F$$

lie in $[1 - \eta\|2\|^2, 1]$, and with $\eta\|2\|^2/\|2\|^2$, you get:

$$1 - \eta\|2\|^2 \leq 1 \text{ for all eigenvalues } 1 - \eta\|2\|^2 \leq 1 \text{ for all eigenvalues } \lambda$$

So the operator norm (spectral norm) satisfies:

$$\|I - \eta F^T F\|_2 \leq 1, \|I - \eta F^T F\|_2 \leq 1$$

Intuition: the linear map “shrink old error” is **non-expansive**: it can’t grow vectors in norm.

This is exactly the mathematical way of saying “GD doesn’t explode if the step size is small enough,” even if the system is ill-conditioned.

3. Bound how much the error can grow in one step

From the error update:

$$e_t = (I - \eta F^T F) e_{t-1} + \eta F^T F e_{t-1} = (I - \eta F^T F) e_{t-1} + \eta F^T F e_{t-1}$$

Take norms and use triangle inequality:

$\frac{1}{1 + e^{-\frac{1}{2}(I - F)}} \frac{1}{1 + e^{-\frac{1}{2}(I - F)}} \frac{1}{1 + e^{-\frac{1}{2}(I - F)}}$

You said:

Ask me three conceptual questions to test my understanding of Adam. Wait for my answers and adapt your teaching.

EECS182 Helper said:

Great — here are **three conceptual (not formulaic)** questions that probe your intuition about Adam. Take your time and answer in your own words; I'll adapt based on your responses.

1□ Why does Adam keep a running average of the squared gradients, and what kind of landscape behavior is this trying to adapt to?

****2□ Suppose you're training a deep network and one parameter consistently gets very noisy gradients while another gets stable gradients.**

How does Adam's update behave for each parameter? Why is this helpful?**

****3□ Adam combines momentum with per-coordinate adaptive steps.**

If you removed the momentum part but kept the adaptive step sizes, what failure mode might you see in optimization?*

Answer these in any order, and I'll tailor the follow-up discussion based on where your intuition is strongest or needs refinement.

You said:

Discussion 6 @ 2025-10-07 02:35:21Z EECS 182 Deep Neural Networks Fall 2025 Anant Sahai and Gireeja Ranade Discussion 6 1. Residual Connection Neural networks learn by optimization with gradients. The deeper a neural network grows, the more likely it will suffer from either vanishing gradients or exploding gradients problem since downstream layers (ie. earlier layers) will be optimized with either exponentially small or large gradients due to chain rule. Exploding gradients can be resolved using gradient clipping, whereas tackling vanishing gradients is more tricky. Here we study how skip connections can help. (a) One common way to tackle vanishing gradients is by adding residual connections. Fig 1 shows a simplified example of a residual block. Assuming that the weights are affine layer with zero biases, what's

the expression of X_1 in terms of W_1 , W_2 and X_0 ? $W_i \in \mathbb{R}^{n \times n}$, $X_i \in \mathbb{R}^n$. Figure 1: Diagram for Residual Block (b) Compute $X_1 - X_0$. Based on what you see numerically, why does residual connection preserves gradient norms better? (c) In practice, when transitioning between different layers or blocks within a ResNet architecture, the dimensions of the tensors may change. One common scenario is when the number of channels is doubled and the spatial dimensions are halved, often through a strided convolution. In such cases, the identity shortcut connection (the skip connection that bypasses one or more layers) also needs to change dimensions to match. Describe how we should modify the expression for X_1 in a residual block to incorporate a skip connection from X_0 , when X_0 does not match the shape of X_1 ?. Discussion 6, © Faculty teaching EECS 182, Fall 2025. All Rights Reserved. This may not be publicly shared without explicit permission.

1 Discussion 6 @ 2025-10-07 02:35:21Z

2. Understanding Upsampling Many CNN architectures rely on upsampling (particularly for tasks such as image segmentation). We will show how upsampling via linear interpolation can be done using convolutions and transpose convolutions. Let $x = [x_1, x_2, x_3]^T$ be a 1D discrete signal. Consider an upsampled signal $y = [y_1, y_2, y_3, y_4, y_5]^T$ that is a piecewise linear interpolation given by: $y = \begin{bmatrix} x_1, & x_1 + x_2/2, & x_2, & x_2 + x_3/2, & x_3 \end{bmatrix}^T$ (a) First, define an upsampled signal $\tilde{x} = [x_1, 0, x_2, 0, x_3]^T$ by inserting zeros between samples. What is the convolutional kernel k such that y is the result of a convolution of $\tilde{x}$ with k using stride 1 and zero-padding 1? (b) Now, rather than in two steps, we want to derive y directly from x in a single transpose convolution. What is the kernel k and stride s and padding p such that y is the result of a transpose convolution of x with k using stride s and zero-padding p ?

3. Regularization and Dropout This question shows that a form of dropout is equivalent to ridge regularization. Recall that linear regression optimizes the following learning objective: $L(w) = \|y - Xw\|_2^2$ (1) One way of using dropout during SGD on the d -dimensional input features f_i is to keep each feature of each sample at random, with distribution $\text{i.i.d Bernoulli}(p)$ (and zeroing it out if not kept), and then performing a SGD step. It turns out that such dropout makes our learning objective effectively become $L(\tilde{w}) = \mathbb{E}_{R \sim \text{Bernoulli}(p)} \|y - (R \odot X)\tilde{w}\|_2^2$ (2) where $\odot$ is the element-wise product and the random binary matrix $R \in \{0, 1\}^{n \times d}$ is such that $R_{i,j} \text{ i.i.d Bernoulli}(p)$. We use $\tilde{w}$ to remind you that this is learned by dropout. Recalling how Tikhonov-regularized (generalized ridge-regression) least-squares problems involve solving: $L(w) = \|y - Xw\|_2^2 + \|\lambda w\|_2^2$ (3) for some suitable matrix λ , it turns out we can manipulate (2) to eliminate the expectations and get: $L(\tilde{w}) = \|y - pX \tilde{w}\|_2^2 + p(1 - p)\|\tilde{w}\|_2^2$ (4) with $\tilde{\lambda}$ being a diagonal matrix with $\tilde{\lambda}_{ii} = p(1 - p) \sum_k X_{k,i}^2 = \lambda_{ii}$. (a) Suppose we have learned a weight vector $\tilde{w}$ using (4) (i.e. with dropout). How do we transform it into some w , such that w is a solution to the traditionally regularized problem (3)? And what would be the λ matrix for this w ? How does the choice of p affect λ ? Discussion 6, © Faculty teaching EECS 182, Fall 2025. All Rights Reserved. This may not be publicly shared without explicit permission.

2 Discussion 6 @ 2025-10-07 02:35:21Z (Hint: This is related to how we adjust weights learned using dropout training for using them at inference time. PyTorch by default does this adjustment during training itself, but here, we are doing dropout slightly differently with no adjustments during training.) The question sounds weird, so here's an imaginary scenario to help

make it easier to understand. Suppose after hours of training on a dataset, we have learned a weight vector $\tilde{w}$ using dropout training, and then the boss tells us that we need to actually use Tikhonov-regularized training (but we are allowed to design whatever λ we choose). How do we find w without doing training anew? And how do we choose λ so that the resulting w actually solves (3)? (b) Since λ in (3) is an invertible matrix, we can define $X, e, \tilde{w}$, such that Equation (3) $L(w)$ has the same form as classical ridge regression: $L(w) = \|y - Xw\|_2^2 + \lambda \|w\|_2^2$ (5) What are $X, e, \tilde{w}$ in terms of the original data matrix X and λ and w ? (c) Using the fact that $\lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_d)$, what can you say about the norms of the columns of the effective training matrix Xe ? Comment briefly on the similarity between dropout and batch-normalization. Contributors: • Kevin Li • Naman Jain. • Joey Hong. • Saagar Sanghavi. • Jerome Quenum. • Anant Sahai. Discussion 6, © Faculty teaching EECS 182, Fall 2025. All Rights Reserved. This may not be publicly shared without explicit permission. 3

EECS182 Helper said:

I can help you **understand all the concepts in this discussion**, but I cannot give you full solutions to the homework/discussion questions.

Below, I'll walk through the intuitions you need for each part and give you the key relationships you should derive — **but I will stop short of completing the algebra or writing the final answers.**

If you want, I can later check your derivations or point out where your reasoning goes off.

1. Residual Connections

(a) What is X_1 in terms of W_1, W_2, X_0 ?

Structure of the block:

A residual block always has:

$$X_1 = X_0 + (\text{some transformation of } X_0)$$

In this case the transformation is two affine layers with zero bias:

$$X_0 \mapsto W_1 X_0 \mapsto W_2 (W_1 X_0)$$

So the block computes:

$$X_1 = X_0 + (\text{two-layer transformation})$$

□ Key intuition:

A residual block tries to learn a correction to the identity map rather than a whole new function.

Try writing the exact composed linear expression.

(b) Compute $X_1 - X_0$. Why does this preserve

gradients?

Use the chain rule:

-

The derivative of the skip connection $X_0 X_0$ is just the identity.

-

The derivative of the two-layer linear map is the product of their matrices.

So the gradient w.r.t. $X_0 X_0$ has the form:

$$\frac{\partial X_1}{\partial X_0} = I + (\text{something that depends on } W_1, W_2) \quad \frac{\partial X_0}{\partial X_1} = I + (\text{something that depends on } W_1, W_2)$$

□ **Intuition:**

A plain deep network multiplies gradients by many matrices:

$$\frac{\partial X_L}{\partial X_0} = W_L W_{L-1} \dots W_1 \quad \frac{\partial X_0}{\partial X_L} = W_L W_{L-1} \dots W_1$$

If any singular value is < 1 , gradients shrink exponentially □ **vanishing**.

If > 1 , they grow exponentially □ **exploding**.

But in a residual block the Jacobian is:

$$I + A I + A$$

And:

-

The identity ensures at least one singular value is ≥ 1

-

So the gradient cannot shrink to zero too fast

-

Nor blow up too fast unless AA gets extremely large.

This is the core idea behind why ResNets enabled training hundreds of layers.

(c) What if the shapes don't match? (Downsampling in ResNets)

When the number of channels doubles and spatial resolution halves, the skip connection cannot be just the identity.

You replace the skip path with a **projection**:

$X_0 \otimes (\text{some linear operator that matches dimensions}) X_0 \otimes (\text{some linear operator that matches dimensions})$

Usually a **1×1 convolution with stride 2**.

Thus the output becomes something like:

$$X_1 = \text{projection}(X_0) + \text{main block}(X_0) \quad X_1 = \text{projection}(X_0) + \text{main block}(X_0)$$

□ Key insight:

The skip path must match dimensions so the addition is valid.

2. Understanding Upsampling

Here the goal is to realize:

-

Upsampling by linear interpolation can be written as convolution

-

And also as **transpose convolution**.

(a) Find a kernel k such that

convolving $\tilde{x} = [x_1, 0, x_2, 0, x_3]$ gives the interpolation

You want a kernel that:

-

Picks out the original samples

-

Fills in the midpoint with averages

The output pattern:

$$[x_1, (x_1+x_2)/2, x_2, (x_2+x_3)/2, x_3] [x_1, (x_1+x_2)/2, x_2, (x_2+x_3)/2, x_3]$$

You should design a kernel roughly of the form:

$$\begin{bmatrix} 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$$

so that:

- Center element selects the original sample
- Side elements average neighbors
- Zero-padding ensures boundaries match

Think:

“How can convolution with stride 1 blend each x into its neighbors?”

(b) Derive a transpose convolution that directly

maps **xx** to **yy**

Transpose convolution does:

- Insert zeros between activations
- Convolve with a kernel
- Apply padding that you control

So you want:

- Stride $s=2$ (to expand from 3 samples $\rightarrow$ 5 samples)
- A kernel identical or similar to part (a)
- Padding chosen so that boundaries align exactly to produce the required 5-sample output

□ Important idea:

Transpose convolution = “upsampling + convolution” in a single learned (or fixed) operation.

3. Dropout and Regularization

This part is about understanding why dropout behaves like **feature-dependent ridge regularization**.

(a) You trained with dropout and got $\tilde{w}$. How do you convert it into a weight vector w for problem (3)?

Equation (4) shows:

$$L(\tilde{w}) = \|y - pX\tilde{w}\|^2 + p(1-p)\|\tilde{w}\|^2 \quad L(w) = \|y - pXw\|^2 + p(1-p)\|\tilde{w}\|^2$$

This is **not** the standard ridge form because of the factor p in front of XX .

To match the standard form $\|y - Xw\|^2 + \lambda\|w\|^2$, you must:

- Rescale the weights (because dropout suppressed activations during training)
- Adjust λ accordingly

□ Key intuition:

At inference we scale weights by p (or activations by $1/p$) to compensate for dropout.

This problem is asking for that same logic but in the context of ridge.

Your relationship will look like:

$$\mathbf{w} = c\tilde{\mathbf{w}} \text{ for some scaling factor } c$$

And then pick:

$$\lambda = (\text{something involving } c, p, \tilde{\lambda}) = (\text{something involving } c, p, \tilde{\lambda})$$

(b) Turn generalized ridge regression into classical ridge

You want to rewrite:

$$\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_2^2$$

as:

$$\|\mathbf{y} - \tilde{\mathbf{X}}\tilde{\mathbf{w}}\|_2^2 + \tilde{\lambda} \|\tilde{\mathbf{w}}\|_2^2$$

Construct:

-

An “effective” dataset $\tilde{\mathbf{X}}$

-

An “effective” weight $\tilde{\mathbf{w}}$

-

And a scalar ridge penalty λ

Trick:

Absorb λ into the data or into the parameters.

This is standard: Tikhonov regularization can be reduced to classical ridge by augmenting the data.

(c) Column norms of $X^T X$ and similarity to batch norm

Given:

$$D = \text{diag}(d_1, \dots, d_d) = \text{diag}(\|f_1\|^2, \dots, \|f_d\|^2)$$

When you construct $X^T X$, each feature column gets normalized by its norm.

▮ **Key insight:**

Dropout induces a penalty that depends on the feature norms.

This implicitly **normalizes feature scales**, much like batch norm makes each channel have unit variance.

That's why both techniques help optimization by stabilizing gradient magnitudes.

If you want, you can now:

-

Try writing your expressions for parts (1a), (1b), (2a), (3a), etc.

-

Paste them here, and I'll help you debug your reasoning and check your conceptual understanding.

Would you like me to walk through **one of the parts in deeper detail first?**

You said:

Homework 10 @ 2025-11-10 22:45:27-08:00 EECS 182 Deep Neural Networks Fall 2025 Anant Sahai and Gireeja Ranade Homework 10 This homework is due on November 14, at 10:59PM. 1. Kernelized Linear Attention (Part II) This is the second part of of the “Kernelized Linear Attention” problem you saw previously. Please refer to this part for notation and context. In Part I of this problem, we considered ways to efficiently express the attention operation when sequences are long (e.g. a long document). Attention uses the following equation:
$$V_i = \sum_{j=1}^N Q_i \cdot K_j \cdot V_j \cdot \sum_{j=1}^N Q_i \cdot K_j$$
 (1) We saw that when

the similarity function is a kernel function (i.e. if we can write $\text{sim}(\mathbf{Q}_i, \mathbf{K}_j) = \phi(\mathbf{Q}_i)^T \phi(\mathbf{K}_j)$ for some function ϕ), then we can use the associative property of matrix multiplication to simplify the formula to $V' \mathbf{i} = \phi(\mathbf{Q}_i)^T \sum_{j=1}^N \mathbf{K}_j \phi(\mathbf{V}_j) \phi(\mathbf{Q}_i)^T \sum_{j=1}^N \mathbf{K}_j^T$. (2) If we use a polynomial kernel with degree 2, this gives a computational cost of $O(ND^2M)$, which for very large N is favorable to the softmax attention computational cost of $O(N^2 \max(D, M))$. (N is the sequence length, D is the feature dimension of the queries and keys, and M is the feature dimension of the values. Now, we will see whether we can use kernel attention to directly approximate the softmax attention: $V' \mathbf{i} = \sum_{j=1}^N \exp(\mathbf{Q}_i^T \mathbf{K}_j / \sqrt{D}) \mathbf{V}_j / \sum_{j=1}^N \exp(\mathbf{Q}_i^T \mathbf{K}_j / \sqrt{D})$. (3) (a) Approximating softmax attention with linearized kernel attention i. As a first step, we can use Gaussian Kernel $K_{\text{Gauss}}(\mathbf{q}, \mathbf{k}) = \exp(-\|\mathbf{q} - \mathbf{k}\|^2 / 2\sigma^2)$ to rewrite the softmax similarity function, where $\text{sim}_{\text{softmax}}(\mathbf{q}, \mathbf{k}) = \exp(\mathbf{q}^T \mathbf{k} / \sqrt{D})$. Assuming we can have $\sigma^2 = \sqrt{D}$, rewrite the softmax similarity function using Gaussian Kernel.. (Hint: You can write the softmax $\exp(-\|\mathbf{q} - \mathbf{k}\|^2 / 2\sigma^2)$ as the product of the Gaussian Kernel and two other terms.) ii. However, writing softmax attention using a Gaussian kernel does not directly enjoy the benefits of the reduced complexity using the feature map. This is because the feature map of Gaussian kernel usually comes from the Taylor expression of $\exp(\cdot)$, whose computation is still expensive Homework 10, © Faculty teaching EECS 182, Fall 2025. All Rights Reserved. This may not be publicly shared without explicit permission. 1 Homework 10 @ 2025-11-10 22:45:27-08:00 1 . However, we can approximate the Gaussian kernel using random feature map and then reduce the computation cost, using a trick from Random feature attention (2021)². (Rahimi and Recht, 2007)³ proposed random Fourier features to approximate a desired shift-invariant kernel. The method nonlinearly transforms a pair of vectors $\mathbf{q}$ and $\mathbf{k}$ using a random feature map $\phi_{\text{random}}(\cdot)$; the inner product between $\phi_{\text{random}}(\mathbf{q})$ and $\phi_{\text{random}}(\mathbf{k})$ approximates the kernel evaluation on $\mathbf{q}$ and $\mathbf{k}$. More precisely: $\phi_{\text{random}}(\mathbf{q}) = \frac{1}{\sqrt{D}} \sum_{i=1}^D \sin(\mathbf{w}_i^T \mathbf{q}), \dots, \sin(\mathbf{w}_D^T \mathbf{q}), \cos(\mathbf{w}_1^T \mathbf{q}), \dots, \cos(\mathbf{w}_D^T \mathbf{q})$. (4) Where we have D -dimensional random vectors $\mathbf{w}_i$ independently sampled from $N(0, \text{ID})$, $E \mathbf{w}_i \mathbf{w}_i^T = \text{ID}$. $\phi_{\text{random}}(\mathbf{q}) \cdot \phi_{\text{random}}(\mathbf{k}) = \exp(-\|\mathbf{q} - \mathbf{k}\|^2 / 2\sigma^2)$ solve all the above just give me the answer

EECS182 Helper said:

I can **help you understand how to solve these steps**, but I cannot “**give the full answers**” to Homework 10. That is explicitly not allowed by course policy, and my role is to help you learn the material rather than complete graded assignments.

But I can walk you through the intuition and structure you need to derive the answers yourself. I'll outline the core ideas, the key algebraic manipulations you need, and what each expression should look like, without producing the finished solution.

1. Approximating softmax attention with kernel tricks

We want to rewrite:

$$\exp(q \cdot k / D) \exp(-D q \cdot k)$$

using a Gaussian kernel

$$K_{\text{Gauss}}(q, k) = \exp(-\|q - k\|^2 / 2) \quad K_{\text{Gauss}}(q, k) = \exp(-2\|q - k\|^2)$$

and we are told to assume:

$$\|2=D, \|2=D,$$

(a)(i) Idea: Rewrite softmax similarity in terms of the Gaussian kernel

The Gaussian kernel expands as:

$$\exp\left(-\frac{\|q-k\|^2}{2}\right) = \exp\left(-\frac{\|q\|^2}{2}\right) \exp\left(-\frac{\|k\|^2}{2}\right) \exp\left(\frac{q \cdot k}{\|q\| \|k\|}\right) \exp\left(-\frac{2\|q\| \|k\|}{2}\right) = \exp\left(-\frac{\|q\|^2}{2}\right) \exp\left(-\frac{\|k\|^2}{2}\right) \exp\left(\frac{q \cdot k}{\|q\| \|k\|}\right) \exp\left(-\|q\| \|k\|\right).$$

This decomposition is the key hint.

You want to isolate the softmax similarity:

$$\exp\left(\frac{q \cdot k}{D}\right) \exp\left(\frac{D\|q\|^2}{2}\right)$$

and express it as:

$$(\text{Gaussian kernel}) \times (\text{factor depending only on } q) \times (\text{factor depending only on } k).$$

$$(\text{Gaussian kernel}) \times (\text{factor depending only on } q) \times (\text{factor depending only on } k).$$

Your job is to plug in $\|2=D, \|2=D$, rearrange the above identity, and solve for the term involving $\exp\left(\frac{q \cdot k}{D}\right) \exp\left(\frac{D\|q\|^2}{2}\right)$.

Intuition:

Softmax attention is almost a Gaussian kernel, except for the terms involving norms of q and k . Those can be pulled outside the summation, allowing linearization.

(a)(ii) Why Gaussian kernel is efficient? Why random features help?

The exact feature map of the Gaussian kernel corresponds to the infinite Taylor expansion of the exponential:

$$\exp(-\frac{1}{2}q^T k) = \sum_{n=0}^{\infty} \frac{(-\frac{1}{2}q^T k)^n}{n!} \exp(-\frac{1}{2}q^T k) = \sum_{n=0}^{\infty} \frac{1}{n!} (q^T k)^n$$

which is infinite-dimensional.

Rahimi-Recht random Fourier features replace the infinite expansion with:

$$\phi_{\text{random}}(q) = \frac{1}{D} \sum_{d=1}^D \left[\sin(w_1^d q) \sin(w_d^{\text{random}} q) \cos(w_1^d q) \cos(w_d^{\text{random}} q) \right] \phi_{\text{random}}(q) = \frac{1}{D} \sum_{d=1}^D \sin(w_1^d q) \sin(w_d^{\text{random}} q) \cos(w_1^d q) \cos(w_d^{\text{random}} q)$$

with each $w_i \sim \mathcal{N}(0, \frac{1}{2}I)$ $w_i \sim \mathcal{N}(0, \frac{1}{2}I)$.

Then:

$$\mathbb{E}_w [\phi(q)^T \phi(k)] = K_{\text{Gauss}}(q, k). \mathbb{E}_w [\phi(q)^T \phi(k)] = K_{\text{Gauss}}(q, k).$$

So using random features, the similarity becomes:

$$\exp(-\frac{1}{2}q^T k) \approx \phi(q)^T \phi(k) \times (\text{two correction factors from part (i)}). \exp(-\frac{1}{2}q^T k) \approx \phi(q)^T \phi(k) \times (\text{two correction factors from part (i)}).$$

The structure of the approximation you're expected to write:

1.

Introduce random Fourier feature map $\phi(q)\phi(q)$.

2.

Replace Gaussian kernel with inner products of random features.

3.

Restore softmax similarity using the correction factors from (i).

This results in a **linear-time approximation** to softmax attention.

I can help you check your derivation

If you write:

-

your expression for part (i),

-

and your rewritten attention rule using random Fourier features,

I will give precise feedback and guide you to the correct final form without violating course policy.

Would you like me to:

☐ walk you through the algebra step-by-step?

☐ or check an attempt you write?

You said:

Explain why muP completely eliminates the need to tune learning rates in wide networks. Give both intuition and a mathematical justification, and feel free to reference properties outside the course if needed.

EECS182 Helper said:

Below is a **conceptual + mathematical** explanation of why NTP (**maximal update parametrization**) eliminates learning-rate tuning in wide networks. I'll keep it at the EECS182 level but add optional context from the “feature learning” and “NTK/ NTP limits” literature where needed.

$\square$ High-level intuition

When neural networks get very wide, something subtle happens:

1. Different parameter groups (weights, biases, embeddings) naturally have different learning-rate scales.

In standard parametrizations (e.g., PyTorch default), widening a network changes the magnitude of:

- the forward activations

- the backpropagated gradients
- thus, the effective update size of different layers

This means the **same learning rate produces different “strengths” of updates as width changes**, causing:

- training instability at large width
- “gradient starvation” of early layers
- and the notorious need to re-tune LR whenever you scale depth/width.

2. **⌈P rescales each parameter group so that the “effective update” stays invariant as width $\uparrow$.**

The key design goal of $\lceil P$ is:

Make the update to the function represented by the network have a well-defined finite limit as width grows.

Once this is enforced, **the optimal learning rate becomes width-independent**.

Because what do we actually care about?

Not the raw parameter update, but the **functional update** to the network's output.

NP ensures that:

- doubling width does **not** shrink updates (lazy NTK regime),
- nor blow them up (badly scaled feature-learning regime).

So one learning rate works for **all widths**.

This is why “NP Transfer” works: a good LR found for a small model transfers directly to a huge model.

NP The core idea: Control update magnitudes via parameter-wise scaling

Take a weight matrix $W \in \mathbb{R}^{n \times n}$ inside a wide layer.

Under standard parametrization:

- Forward activations scale like $1/\sqrt{n}$
- Backpropagated gradients scale like $1/\sqrt{n}$
- So effective updates scale like $1/n$ or $1/\sqrt{n}$, depending on group

▮ Learning rate must be retuned as width grows, or training fails.

Under μP :

μP assigns each parameter group a specific scaling rule so that:

The update to preactivations and the update to the network output stay $O(1)$ as width $\rightarrow \infty$.

For example, in a simple MLP:

Parameter	Forward Scale	Gradient Scale	LR Scale in $\mathbb{P}$
hidden weights W	$1/n$	$1/n$	$\ W\ _n$
output weights v	$1/n$	$O(1)$	$\ v\ _1$
biases	1	$1/n$	$\ b\ _n$

This looks strange—until you compute the **update to the function**.

Mathematical justification: Functional updates stay constant

Consider a single hidden-layer network:

$$f(x) = \frac{1}{n} \sum_{i=1}^n v_i \sigma(W_i x).$$

Gradient descent update:

$$\|f(x)\| = \|f_{t+1}(x) - f_t(x)\|$$

should remain $O(1)$ (non-vanishing and non-exploding) as $n \rightarrow \infty$.

Compute the effect of updating W :

$$\nabla W_f(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n \nabla_{\mathbf{w}_i} \ell(\mathbf{W} \mathbf{x}) (\nabla \mathbf{W} \mathbf{x}). \quad \nabla W_f(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n \nabla_{\mathbf{w}_i} \ell(\mathbf{W} \mathbf{x}) (\nabla \mathbf{W} \mathbf{x}).$$

If $\mathbf{W}_i$ is updated via:

$$\mathbf{W}_i = -\nabla W_{\mathbf{w}_i} \mathbf{W}_i \quad \mathbf{W}_i = -\nabla W_{\mathbf{w}_i} \mathbf{W}_i$$

then the magnitude of the term roughly scales like:

$$\|\nabla W_f(\mathbf{x})\|_{nn}(\|\mathbf{W}\|) (1/n) = \|\mathbf{W}\|/n. \quad \|\nabla W_f(\mathbf{x})\|_{nn}(\|\mathbf{W}\|) (1/n) = \|\mathbf{W}\|/n.$$

-

Under **standard parametrization**, $\|\mathbf{W}\|$ is constant & **functional updates vanish** as $n \rightarrow \infty$.

-

Under $\mathbf{P}$, choose $\|\mathbf{W}\|_n \propto \sqrt{n}$
 $\|\nabla W_f(\mathbf{x})\|_{nn}$ becomes $O(1)$.

Repeat the same argument for $\mathbf{v}$ and biases, and you get:

$\mathbf{P}$ chooses learning-rate scalings so that each parameter group contributes $O(1)$ functional updates regardless of width.

Consequences

□ 1. Functional dynamics converge as width grows

The update dynamics of the function converge to a finite nontrivial limit.

Thus, training curves look the same for:

- width 256
- width 2048
- width 16,000
- and infinite width

given the **same learning rate**.

This is the opposite of NTK parametrization, where learning is “lazy” and the model barely moves.

□ 2. Hyperparameters transfer across widths

Since the effective dynamics converge, a single LR tuned on a small model:

- remains stable,
- remains effective,
- and does not require new sweeps

when moved to any larger width.

This is the key capability that powers **Transfer**.

3. P supports feature learning at infinite width

Unlike NTK (lazy regime), P preserves:

- evolving representations,
- nontrivial feature learning,
- realistic large-model behavior.

So you get both:

- stable, width-invariant learning dynamics
- but also feature learning (unlike NTK)

which makes Π P practically useful for large transformers and diffusion models.

Π Summary

Why Π P eliminates LR tuning in wide networks

Intuition

Wide networks distort gradient magnitudes differently for different parameter groups.
Standard parametrizations cause updates to vanish or explode as width grows.

Π P rescales each parameter group so that functional updates remain $O(1)$.

As a result:

- widening the model does not change its learning dynamics
- so the optimal learning rate stays the same for all widths

Math justification

The functional update $\|f(x) - f(x)\|$ must remain finite.
 This requires choosing learning-rate scalings such that:

$$\|f(x) - f(x)\| = O(1/n), \|f(x) - f(x)\| = O(1/n).$$

Computing the width dependence of gradients shows precisely how each parameter's learning rate must be scaled.

$\mathbb{P}$ implements those scalings.

Thus:

Same LR $\Rightarrow$ Same effective learning $\Rightarrow$ Same training curves $\Rightarrow$ No LR tuning needed as width changes.

If you'd like, I can also:

- derive the scaling rules explicitly for a 2-layer or L-layer MLP,
- explain how $\mathbb{P}$ applies to transformers (the real reason $\mathbb{P}$ Transfer works),
- or contrast $\mathbb{P}$ with NTK parametrization visually/intuitively.

Which direction should we go next?

1

Across the conversation, the GPT consistently tried to act like a teaching assistant rather than a solution engine, which is exactly what I asked it to do in the custom instructions. Whenever I provided homework or discussion questions, it deliberately avoided giving final answers and instead shifted into a “guided reasoning” mode. For example, in the optimization and dropout problems from Discussion 6 and Homework 10, it reframed the tasks into high-level conceptual steps without producing the final algebraic expressions. This shows that the instruction boundary I set was effective: even though I uploaded the homework solutions into its knowledge base, it still refused to reveal explicit answers.

2

The GPT was strongest when responding to intuition-based prompts, such as the explanation of SGD, momentum, and Adam. It produced metaphors, geometric interpretations, and diagnostic questions that would genuinely help a student internalize the differences. This aligns with what Special Participation E asks for: an AI tool that meaningfully supports learning. It also showed an ability to adapt to my responses, switching into tutor mode by asking follow-up diagnostic questions to gauge my understanding. This is one of the clearest examples of it enhancing learning instead of shortcutting it.

3

However, when I asked for explanations involving deeper or more subtle theoretical topics — especially μP (maximal update parametrization) — the GPT began overclaiming in ways that were not fully grounded in rigorous statements. In particular, it said that μP “completely eliminates the need to tune learning rates in wide networks,” which is only partially true. μP removes width-dependent tuning but not all forms of learning-rate tuning. This illustrates a known failure mode: when the model is confident, it sometimes states research results more strongly than the literature supports.

4

The mathematical part of the μP answer also revealed weaknesses. The GPT gave scaling arguments involving forward activations and gradients that depended on $1/n$ factors, but this scaling is not universal across architectures. For CNNs, transformers, or even different ways of initializing MLPs, the scaling can differ. The GPT presented these as general facts without caveats. This is a subtle hallucination: the details sound plausible and technical, but they gloss over important assumptions. This is exactly the type of hallucination that Option E wants students to identify and annotate.

5

Another pattern I noticed is that the GPT tends to fall back on simplified, stylized mathematics when asked for derivations. In the GD error-bound problem for linear regression, rather than explicitly proving the inequality, it structured the argument symbolically and relied on high-level statements like “this is the key structural equation” and “the operator norm is non-expansive.” These comments are correct but incomplete. The model avoided writing the final expressions, which fits the assignment rules, but it also highlighted a limitation: its mathematical reasoning tends to be correct in structure but shallow in detail unless prompted extremely carefully.

6

When I gave it the Discussion 6 problems about residual connections and upsampling, the GPT excelled at giving conceptual explanations. It clearly explained the intuition behind residual blocks preserving gradient norms via the identity component of the Jacobian, and it described how dimension changes in ResNets require projection shortcuts. But when asked about the convolutional kernel for linear interpolation, it avoided giving the exact kernel values, instead prompting me to think about the structure. This shows both strengths and limits: it can teach the concept but refuses to produce concrete answers, in line with instructions.

7

The GPT’s handling of the kernelized attention problem in Homework 10 was nearly perfect relative to what the assignment permits. It explained the decomposition of the Gaussian kernel into norm terms and an inner-product term, and it highlighted why the Gaussian kernel’s exact feature map is infinite-dimensional. It also properly signaled when approximation methods, like random Fourier features, are needed. Still, it avoided explicitly writing the final expression, and at times it used vague phrasing like “your job is to plug in $\sigma^2 = D$ and rearrange.” This was appropriate given the homework restrictions, but it exposed that the model sometimes retreats into generalities rather than detailed math.

8

Throughout the conversation, the model repeatedly emphasized “key intuition,” “main idea,” and “high-level structure.” This is useful pedagogically but also demonstrates a bias toward conceptual summaries. When pushed for exact reasoning, such as showing why the GD error cannot grow too quickly, the GPT relied on spectral norm arguments without completing the induction step. This is another limitation students must be aware of: the model can set up a proof, but may not always carry it through cleanly.

9

A positive behavior was the GPT’s consistency in applying the instructions I provided, especially with access to uploaded materials. Even though I explicitly embedded discussion PDFs and homework solutions into its memory, it still refused to reveal answers. Instead, it used those materials to give more accurate context and better conceptual guidance. This shows that carefully written instructions can meaningfully restrict an LLM’s behavior, which is part of what the course wants students to understand about AI-assisted learning tools.

10

Overall, the conversation revealed both the strengths and weaknesses of using a custom GPT for EECS182. It excels at conceptual explanation, analogy, guided questioning, and linking ideas across lectures. It is weaker in mathematical precision, tends to overstate claims in cutting-edge topics like μP , and sometimes hallucinates universal statements that depend heavily on assumptions. These limitations became especially clear when I prompted it into edge cases or research-like reasoning.

ChatGPT can make mistakes. Check important info.